Изучаем Claude «на 200%»: подробный разбор бесплатной 4-часовой лекции Anthropic

> Статья-разъяснение к посту канала «Доктор GPT» (t.me/doktorgpt/3119). Первоисточник — бесплатный четырёхчасовой курс-лекция от инженеров Anthropic о промптинге Claude и построении ИИ-агентов (YouTube, ~3 ч 46 мин, видео `QRlkSPF87jQ`).

О чём вообще речь

В конце сентября 2026 года Anthropic выложила в открытый доступ полный четырёхчасовой курс о том, как «выжимать максимум» из Claude. Это не маркетинговая презентация, а рабочий материал: инженеры компании показывают, как они сами ежедневно используют собственную модель, какие ошибки совершают пользователи и какие приёмы реально улучшают качество ответов.

Пост «Доктора GPT» выделяет четыре ключевых блока лекции:

Таймкод	Тема
00:15	Как правильно писать запросы (промпты) в Claude
33:21	Что заставляет Claude «тупить» и писать неэффективный код
01:33:39	Как инженеры Anthropic используют Claude каждый день
02:50:56	Один «мощный фикс», который заметно прокачивает мышление Claude

В расширенных тайм-кодах курса встречаются ещё два важных пункта, которые стоит знать:

Таймкод	Тема
11:41	«Контракт выхода» (Output Contract) — лекарство от непоследовательных ответов
30:04	Агентный цикл (agent loop), который делает агентов на Claude надёжными

Ниже — подробное разъяснение каждого блока: что имеется в виду, почему это работает и как применить на практике. Разбор опирается на сам пост, описание курса и официальные руководства Anthropic по промпт-инжинирингу и Claude Code.

Блок 1 (00:15). Как правильно писать запросы в Claude

Главный тезис лекции: **модель не умеет читать мысли**. Большинство «тупых» ответов — следствие того, что в запросе не хватает информации, которую автор держал в голове, но не написал. Anthropic предлагает думать о промпте как о задании для нового сотрудника: толкового, но ничего не знающего о вашем контексте.

Принципы, которые разбирают в лекции

1. Явность вместо намёков. Вместо «сделай получше» — конкретика: что именно улучшить, по каким критериям, что считается успехом. Claude буквально исполняет то, что написано, а недосказанное додумывает — и не всегда так, как вы хотели.

2. Контекст — до задачи, а не после. Кто вы, для кого результат, какие ограничения (объём, стиль, формат, стек технологий). Модель «читает» промпт один раз и строит ответ на его основе, поэтому важные условия в конце или «в следующем сообщении» работают хуже, чем всё и сразу.

3. Структура запроса. Длинные промпты стоит размечать: отделять инструкции от данных, данные — от примеров. Anthropic рекомендует XML-подобные теги (`<instructions>`, `<data>`, `<example>`), потому что Claude специально обучен их распознавать. Это убирает классическую ошибку, когда модель начинает «выполнять» текст, который был просто исходными данными.

4. Примеры (few-shot). Один-три примера «вход → желаемый выход» дают больше, чем абзац словесных описаний. Модель отлично обобщает по образцу: формат, тон, уровень детализации.

5. Роль. Указание роли («ты — senior backend-разработчик, пишешь ревью для джуна») смещает стиль и глубину ответа в нужную сторону.

11:41 — Output Contract: «контракт выхода»

Отдельный приём, который в лекции подают как лекарство от непоследовательных ответов. Идея: в промпте жёстко фиксируется **контракт на формат результата** — как в API:

```
Ответ верни строго в таком виде:
1. Раздел «Диагноз» — 2–3 предложения.
2. Раздел «Решение» — нумерованные шаги.
3. Раздел «Риски» — маркированный список, не более 5 пунктов.
Никакого текста до и после этих разделов.
```

Зачем это нужно:

- **Воспроизводимость.** Один и тот же промпт в разные дни даёт одинаково устроенные ответы — их можно сравнивать, парсить, вставлять в пайплайн.
- **Меньше «воды».** Модель не растекается вступлениями и извинениями, потому что контракт их не предусматривает.

- **Автоматическая проверка.** Если ответ должен быть JSON с конкретными полями, его легко проверить кодом и перезапросить при нарушении.

Практическое правило: любой промпт, который вы используете больше одного раза, должен заканчиваться контрактом выхода.

Блок 2 (33:21). Почему Claude «тупит» и пишет неэффективный код

Самый прикладной раздел для разработчиков. Лекция разбирает типовые причины, по которым Claude «застревает» на задачах с кодом, — и почти все они на стороне пользователя, а не модели.

Причина 1. Размытая постановка задачи

«Почини баг» или «сделай быстрее» без описания ожидаемого поведения, воспроизведения и критериев готовности. Модель начинает гадать и чинит не то. Решение: формулировать задачу как мини-тикет — что сломано, как воспроизвести, что должно получиться, как проверить.

Причина 2. Замусоренный контекст

Контекстное окно — это рабочая память модели, и она не бесконечная. Если в диалоге накопились десятки сообщений, неудачные попытки, простыни логов и куски нерелевантного кода, модель «размазывает внимание» и начинает путаться в собственных старых решениях. Решения:

- начинать новую сессию для новой задачи, а не тянуть один бесконечный чат;
- давать только релевантные файлы и фрагменты, а не весь репозиторий;
- просить модель сначала составить краткое резюме контекста и плана, а потом работать от него.

Причина 3. Модель не знает соглашений проекта

Claude пишет «усреднённый» код, если ему не рассказали про ваши стандарты: стиль, выбранные библиотеки, запрещённые паттерны, команды сборки и тестов. В Claude Code для этого есть файл `CLAUDE.md` в корне проекта — постоянная памятка, которую агент читает в начале каждой сессии. Туда кладут: команды (`build`, `test`, `lint`), архитектурные соглашения, «грабли» проекта.

Причина 4. Код без плана

Если попросить «напиши фичу» сразу, модель начнёт генерировать код с первой строки и по ходу загонит себя в неудачную архитектуру. Лекция (и официальные гайды Anthropic) рекомендуют двухфазный режим: **сначала план, потом код**. Попросите Claude исследовать код и предложить план, утвердите или поправьте его — и только затем давайте команду писать.

30:04 — агентный цикл, который делает агентов надёжными

Для автономных агентов лекция формулирует базовый цикл:

```
собрать контекст → действовать → проверить результат → исправить → повторить
```

Ключевое слово — «проверить». Надёжный агент не считает задачу решённой после генерации кода: он запускает тесты/линтер/скрипт проверки, видит ошибку, чинит и прогоняет проверку снова. Именно этот цикл «действие → верификация → ремонт» отличает агента, которому можно доверить задачу целиком, от «генератора кода с первой попытки». Если вы строите своего агента — заложите в него явный шаг проверки с объективным сигналом (выход тестов, код возврата, сравнение с эталоном), а не самооценку модели.

Блок 3 (01:33:39). Как инженеры Anthropic используют Claude каждый день

Этот раздел — взгляд «на кухню»: какие рабочие процессы с Claude прижились внутри самой Anthropic.

Типовые сценарии:

- **Онбординг в незнакомый код.** Новичок не читает репозиторий неделями, а спрашивает Claude: «как устроена авторизация?», «где обрабатывается этот запрос?». Модель с доступом к коду отвечает со ссылками на файлы.
- **Explore → Plan → Code → Commit.** Канонический workflow Claude Code: агент сначала изучает код, затем пишет план, затем реализует, затем коммитит с осмысленным сообщением. Каждую фазу можно контролировать.
- **Разработка через тесты (TDD).** Сначала просят Claude написать тесты на желаемое поведение, убеждаются, что они падают, а потом просят писать реализацию, пока тесты не позеленеют. Тесты становятся тем самым объективным сигналом проверки из агентного цикла.
- **Рутина:** миграции, рефакторинги по шаблону, починка линтера, документация, черновики ревью — задачи, где важна аккуратность, а не креативность, отдаются агенту почти полностью.
- **Параллельные субагенты.** Большая задача режется на независимые куски, каждый кусок выполняет отдельный экземпляр агента, человек сводит результаты.

Метрики, по которым Anthropic судит об эффекте (упоминаются в описании курса): скорость онбординга новых инженеров, длительность цикла PR (от открытия до мержа), доля коммитов, сделанных с помощью Claude. Полезный ориентир, если вы внедряете ИИ-инструменты в своей команде: мерить стоит сквозные показатели процесса, а не «количество сгенерированных строк».

Блок 4 (02:50:56). «Один мощнейший фикс», который прокачивает мышление Claude

Интрига поста — простая настройка, которая «делает Claude гораздо умнее». Речь о **расширенном мышлении (extended thinking)** и приёмах, которые заставляют модель рассуждать перед ответом, а не выпаливать первое приближение.

Что это такое

По умолчанию модель стремится сразу выдать ответ. В режиме расширенного мышления Claude сначала «думает» — строит внутреннюю цепочку рассуждений, разбирает задачу, проверяет гипотезы — и только потом формулирует итог. Для сложных задач (архитектура, отладка, математика, многошаговые планы) это даёт качественно другой результат.

Как включить

- **В приложении/API:** включить extended thinking (в API — параметр `thinking` с бюджетом токенов на рассуждения).
- **В Claude Code:** волшебные слова в промпте — `think` < `think hard` < `think harder` < `ultrathink`. Каждая ступень выделяет модели больший бюджет на размышления перед действием.
- **В обычном промпте без спецрежима:** попросить явно — «Сначала разбери задачу по шагам в блоке , перечисли варианты и их риски, и только после этого дай финальный ответ».

Вторая половина «фикса»: самопроверка (self-check)

В лекции приём подаётся в связке «метод + самопроверка»: после решения модель просят **проверить собственный ответ** — отдельным шагом или отдельным запросом:

```
Теперь проверь своё решение: пройди по каждому требованию из задачи и
отметь, выполнено ли оно. Найди минимум два слабых места своего ответа.
Если нашёл ошибку — исправь и выведи финальную версию.
```

Почему это работает: генерация и критика — разные режимы. Модель, которая только что написала код с ошибкой, в режиме проверки эту ошибку часто находит, потому что смотрит на текст «свежим взглядом», как ревьюер. Этот же механизм снижает и галлюцинации, о которых говорится в посте: прямое разрешение «если не уверен или данных недостаточно — так и скажи» плюс просьба перепроверить факты заметно сокращают выдуманные ответы.

Цена вопроса

Расширенное мышление тратит дополнительные токены, то есть деньги и время. Рекомендация из лекции совпадает со здравым смыслом: включать его для сложных

задач (отладка, проектирование, анализ), выключать для рутины (переформатирование, простые правки). Экономия токенов, упомянутая в посте, достигается и другими способами: короткий релевантный контекст вместо «вывалить всё», контракт выхода (без «воды» ответ короче), новая сессия вместо гигантской истории.

Сводный чек-лист: применяем лекцию за 10 минут

1. **Пишите промпт как тикет:** контекст → задача → ограничения → критерии готовности → формат ответа.
2. **Фиксируйте контракт выхода** для любого повторно используемого промпта.
3. **Размечайте длинные промпты** тегами: инструкции отдельно, данные отдельно, примеры отдельно.
4. **Давайте 1–3 примера** желаемого результата вместо длинных описаний.
5. **Для кода:** заведите `CLAUDE.md` с командами и соглашениями проекта; сначала план, потом реализация; новая задача — новая сессия.
6. **Для агентов:** встройте цикл «действие → объективная проверка → исправление»; не доверяйте самооценке модели без тестов.
7. **Для сложных задач:** включайте extended thinking (`think hard` / `ultrathink` в Claude Code) и добавляйте шаг самопроверки.
8. **Против галлюцинаций:** разрешите модели отвечать «не знаю» и просите перепроверять утверждения по источникам.
9. **Меряйте эффект** сквозными метриками: время онбординга, цикл PR, доля задач, закрытых с ИИ, — а не числом сгенерированных строк.

Где смотреть и что почитать

- **Сама лекция:** YouTube, видео `QRlkSPF87jQ` (~3 ч 46 мин), бесплатно. Можно слушать фоном, но блоки 33:21 и 02:50:56 стоит смотреть внимательно.
- **Исходный пост:** t.me/doktorgpt/3119.
- **Официальные материалы Anthropic по теме:**
- руководство по промпт-инжинирингу в документации Claude (platform.claude.com/docs) — явность, XML-теги, примеры, chain of thought;
- статья «Claude Code: Best practices for agentic coding» — `CLAUDE.md`, план-режим, TDD, уровни `think`;
- курсы Claude Academy (academy.claude.com) — бесплатные структурированные курсы, включая «Claude Code 101» и «Building with the Claude API».

Вывод

Главная мысль четырёхчасовой лекции умещается в одну фразу: **качество ответа Claude — это в первую очередь качество постановки задачи и процесса, а не «ума» модели**. Чёткий промпт с контрактом выхода, чистый контекст, план перед кодом, объективная проверка результата и включённое расширенное мышление на сложных задачах — пять приёмов, которые закрывают большинство жалоб на то, что модель «тупит», «лжёт» и «пишет плохой код». Всё это доступно любому пользователю уже сейчас и не требует ни платных курсов, ни специальных инструментов.